Kafka Streams

Data processing and transformation library within kafka. It is a Java library. No need to create a separate cluster. Highly scalable, elastic and fault tolerant. Exactly once capability, one record processed at a time.
It sits on the right side of the kafka cluster. Then it transforms and sends data.

Stream: Sequence of records
Stream Processor: Node in processor graph. Transforms incoming streams record by record and may create a new stream from it.
Topology: Graph of processors chained together by streams.
Source Processor: Takes data directly from the topic. Doesn't transform the data.
Sink Processor: Does not have children, sends stream data to kafka topic

To create a topology,

1. Stream from kafka
2. MapValues / flatmap can do multiple time
3. SelectKey to apply key
4. GroupByKey before aggregation
5. Do final operation/aggregation before sending to stream
6. Create stream to write to kafka and send

```
public Topology createTopology(){
    StreamsBuilder builder = new StreamsBuilder();
    // 1 - stream from Kafka

    KStream<String, String> textLines = builder.stream("word-count-input");
    KTable<String, Long> wordCounts = textLines
            // 2 - map values to lowercase
            .mapValues(textLine -> textLine.toLowerCase())
            // can be alternatively written as:
            // .mapValues(String::toLowerCase)
            // 3 - flatmap values split by space
            .flatMapValues(textLine -> Arrays.asList(textLine.split("\\W+")))
            // 4 - select key to apply a key (we discard the old key)
            .selectKey((key, word) -> word)
            // 5 - group by key before aggregation
            .groupByKey()
            // 6 - count occurences
            .count(Materialized.as("Counts"));

    // 7 - to in order to write the results back to kafka
    wordCounts.toStream().to("word-count-output", Produced.with(Serdes.String(), Serdes.Long()));

    return builder.build();
}
```

We need to tell kafka that the type of values have changed while returning so we use Serdes.toString() and Long to tell the data type.
Then to start,

```
KafkaStreams streams = new KafkaStreams(wordCountApp.createTopology(), config);
streams.start();
```

The topology represents all the streams and processors of streams application.

To gracefully shutdown the streams application, Make sure below is last line of the code.
Runtime.getRuntime().addShutdownHook(new Thread(streams::close));

Internal Topics:
Running a kafka stream may eventually create internal intermediary topics. These can be-
- Repartitioning topics: When we start transforming key of stream
- Changelog topics: in case of aggregated operations, streams save compacted data

They are managed by kafka streams. Used to store / re store state and re-partition data.
These are prefixed by application.id parameter.

KStreams:
They are all inserts similar to a log. It is an infinite sequence of messages.
If same key comes again, it is added to stream next. Old one stays as it is.

KTable:
They are all upserts on non null values. Deletes on null values. It is similar to table and parallel with log compacted topics.
For a key value is added. If same key comes again it is updated. If null value comes, it is deleted.
Log compaction works best for KTable.

Use KStream when
- Reading from a topic that is not compacted
- If new data is partial info/ transactional

Use KTable when
- Reading from topic that's log compacted (aggregated)
- If we need a structure like db table where every update is self sufficient

Global KTable:
Similar to KTables. It reads data from all partitions of a topic across all instances of an application whereas KTable reads from only one partition of a topic.
It can be used to broadcast info to all instances of an application.

Stateless vs stateful operations

MapValues vs Map: Both take one record and produce another record. The differences are-
- MapValues only affects values whereas Map affect both key and values
- == does not change keys and doesn't trigger repartition whereas Map triggers repartition.
- MapValues can be used for both KStream and KTable where Map for only KStream

Filter vs FilterNot: Takes one record and produces one or zero record.

- Filter doesn't change keys / values and doesn't trigger repartition. Same filterNot.

FlatMapValues vs FlatMap: Takes one record and produces zero, one or more records. Can be used for KStreams only.

- FlatMapValues doesn't change keys whereas FlatMap changes keys.
- == doesn't trigger repartition for FlatMapValues whereas FlatMap == triggers repartition.

Branch: Splits Kstream based on multiple predicates. To use,
KStream<String, Long>[] branches = stream.branch((key,value)-> value > 100, (key,value)-> value >10, (key,value)-> value > 0);
Predicates are evaluated in order.

SelectKey: assigns new key to record from old key, value data. == marks the data for repartitioning. Best to isolate the transformation to know exactly where partitioning happens.

Reading data from Kafka:
We can read a topic as a KStream, KTable, GlobalKTable. To read,
KStream<String, Long> stream = builder.stream(Serdes.String(), Serdes.Long(), "topic")
KTable<String, Long> table = builder.table(Serdes.String(), Serdes.Long(), "topic")
GlobalKTable<String, Long> stream = builder.globalTable(Serdes.String(), Serdes.Long(), "topic")

Writing to kafka:

- Using to -> stream.to("topic"), table.to("topic"). Terminal operation doesn't return any.
- Using through -> stream.through("topic"). Writes data and gets back a stream/table from the topic.

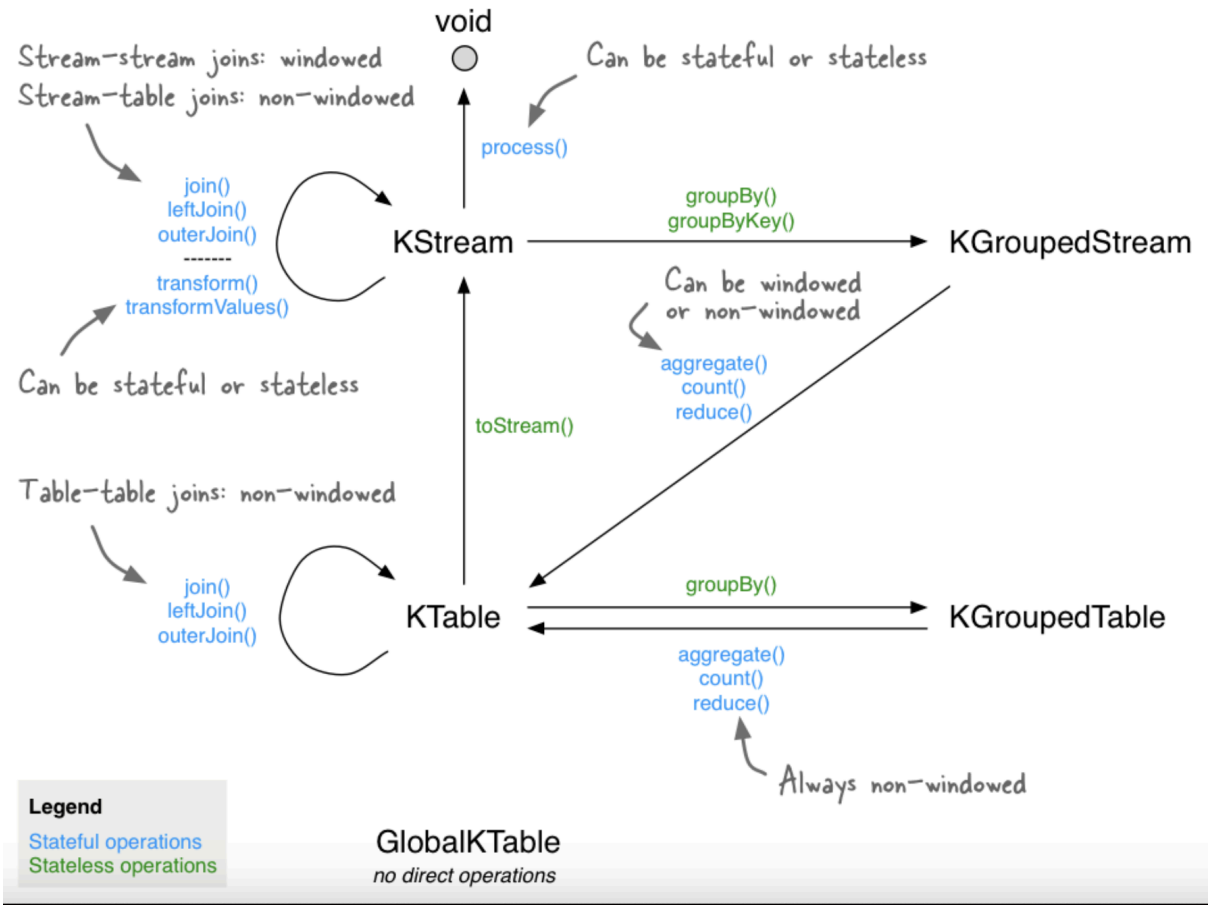
Streams marked for re-partition: As soon as an operation changes key, stream will be marked for repartition. It incurs a performance cost.
When a key is changed, data needs to be shuffled around (spark streaming) which incurs a network cost.

Duality of KStream and KTable:
A stream can be considered a changelog of table where each record captures a state change on table.
Table can be considered as a snapshot of latest key, value in a stream.

Transform KStream into KTable and vice versa:
KStream<byte[], String> stream = table.toStream();
To change to table, use groupByKey() and aggregation step like count().
Write to kafka as intermediate topic and read as a KTable.

KStreams and KTables other operations (Stateful):

- KTable groupBy -> helps perform aggregations. Triggers repartition since key changes. Returns KGroupedTable by using table.groupBy(...)
- KGroupedStream, KGroupedTable Count -> Count counts the number of records by grouped keys. Null keys and values are ignored on KStreams but in KGroupedTables, only null keys are ignored and null values are treated as delete.
- KGroupedStream, KGroupedTable Aggregate -> for KGroupedStream, we need an initializer, an adder, serde and a state store name. For KGroupedTable, we need an initializer, an adder, a subtractor, serde and a state store name.
- KGroupedStream, KGroupedTable Reduce -> Simplified version of aggregate. Result type has to be same as input.
- KStream Peek: Allows to apply side-effect operation to a KStream like printing, collect statistics and get the same KStream as result. Can be executed multiple times.
- KStream Transform/TransformValues



Exactly Once Semantics:

Exactly once means that data processing on each message will happen only once and pushing back to kafka also only once (kafka performs de-duplication).
To solve the problems of a message being processed multiple time, we can now

- Make idempotent producers
- We can write multiple messages to different kafka topics as part of 1 transaction.

To do this, in properties of stream,

```
config.put(StreamsConfig.PROCESSING_GUARANTEE_CONFIG,
StreamsConfig.EXACTLY_ONCE_V2);
```

This may increase latency, but we can fine tune by commit.interval.ms

Joins in Kafka Streams:

Joining means taking a KStream/KTable and creating a new KStream/KTable from it.

Join operands	**Type**	**(INNER) JOIN**	**LEFT JOIN**	**OUTER JOIN**
KStream-to-KStream	Windowed	Supported	Supported	Supported
KTable-to-KTable	Non-windowed	Supported	Supported	Supported
KTable-to-KTable Foreign-Key	Non-windowed	Supported	Supported	Not Supported
KStream-to-KTable	Non-windowed	Supported	Supported	Not Supported
KStream-to-GlobalKTable	Non-windowed	Supported	Supported	Not Supported
KTable-to-GlobalKTable	N/A	Not Supported	Not Supported	Not Supported

These 3 joins - KStream - KStream, KTable-KTable, KStream-KTable can only happen if data is co-partitioned otherwise Stream will fail with Runtime Error.
That means that the number of partitions should be the same on stream and table.
To co-partition data, if no of partitions are different, then write back topics through kafka before join. This has network cost.

To remove this, we can use GlobalKTable. If data is small enough to be on all applications, we can read as GlobalKTable. Using this, we can join if the number of partitions are not same.